

HTMX Documentation for Your Project

HTMX helps you update parts of a web page without reloading the entire page. It works by sending small requests to the server and swapping the returned HTML into the page. This makes your UI feel faster and smoother while keeping the server logic simple.

Dynamic Content Loading

You can fetch and show content automatically when the page or element loads.

```
<div hx-get="/Dashboard/RecentOrders"
      hx-trigger="load"
      hx-swap="innerHTML">
  Loading...
</div>
```

This triggers a GET request on load and replaces the element's inner content.

Search with Debouncing

Use a short delay to avoid sending a request for every keystroke.

```
<input type="text"
       name="query"
       hx-get="/Sales/Search"
       hx-trigger="keyup changed delay:300ms"
       hx-target="#sales-table">
```

The request fires only after typing stops for 300ms.

Form Submissions

Forms can submit through HTMX without refreshing the page.

```
<form hx-post="/Sales/Create" hx-target="#main-content">
  <!-- form fields -->
```

```
<button type="submit">Save</button>
</form>
```

The response replaces the content inside `#main-content`.

Delete with Confirmation

HTMX can ask for confirmation before making a dangerous request.

```
<button hx-delete="/Sales/Delete/123"
        hx-confirm="Are you sure you want to delete this?">
  Delete
</button>
```

Common HTMX Attributes

Attribute	Purpose	Example
hx-get	Send a GET request	hx-get="/api/data"
hx-post	Send a POST request	hx-post="/api/save"
hx-delete	Send a DELETE request	hx-delete="/api/delete/1"
hx-target	Where to place the response	hx-target="#result"
hx-swap	How to replace content	hx-swap="innerHTML"
hx-trigger	Event that fires the request	hx-trigger="load"
hx-push-url	Update browser URL	hx-push-url="true"
hx-confirm	Show a confirmation message	hx-confirm="Are you sure?"

Swap Options

- **innerHTML** — Replace only the inside of the element
- **outerHTML** — Replace the entire element
- **beforebegin** — Insert before the element
- **afterbegin** — Insert at the start inside the element
- **beforeend** — Insert at the end inside the element
- **afterend** — Insert after the element
- **delete** — Remove the element

- **none** — Do not swap anything
-

ASP.NET Core Pattern

In your `.cshtml` file:

```
<div hx-get="/YourPage/YourHandler" hx-trigger="load">
    Loading...
</div>
```

In your `.cshtml.cs`:

```
public IActionResult OnGetYourHandler()
{
    return Partial("_YourPartialView");
}
```

Partial view `_YourPartialView.cshtml`:

```
<div>
    Your dynamic content here
</div>
```

Pages to Complete

- Purchases
- Stock
- Parties
- Reports
- Invoices
- Accounts
- Settings

Next steps include connecting your database, replacing static HTML with live data, securing your routes, and running:

```
dotnet build
dotnet run
```

Tips

- Keep partial views focused and clean.
 - Show a small loading message while waiting for responses.
 - Handle errors with `hx-on::after-request`.
 - Use delays on search fields to reduce server requests.
 - Use `hx-push-url` when you want page history.
-

CRUD Example

List

```
<table>
  <tbody hx-get="/Items/List" hx-trigger="load">
    Loading...
  </tbody>
</table>
```

Create

```
<button hx-get="/Items/Create" hx-target="#modal">
  Add New
</button>
```

Edit

```
<button hx-get="/Items/Edit/123" hx-target="#modal">
  Edit
</button>
```

Delete

```
<button hx-delete="/Items/Delete/123"
  hx-confirm="Delete this item?"
  hx-target="closest tr"
  hx-swap="outerHTML">
  Delete
</button>
```

This document can be expanded or exported if needed. Let me know if you want a PDF version, styling, or formatting changes.